Retrieval-Augmented Generation in Finance: A Comparative Evaluation of Vector, Hierarchical, and Graph-Based RAG Models

Kexin Deng[1], Yueying Wang[1], Yuewei Wang[1], Yun Zhang[1], Yihan Zhao[1], George Lin[1], Yu Yu[2], Charles Zhou[2], Sasha Stoikov[1], Laura Appleby[1],

[1]Cornell University, Ithaca, NY, United States
[2]BlackRock, Inc., New York, NY, United States
Email: kd537@cornell.edu, yw2698@cornell.edu, yw2695@cornell.edu, yz2847@cornell.edu, yz3337@cornell.edu, jl2969@cornell.edu, Yu.Yu@blackrock.com, charles.zhou@blackrock.com sashastoikov@gmail.com lla32@cornell.edu

February 2026

Abstract

Retrieval-augmented generation (RAG) systems have emerged as a powerful paradigm for enterprise knowledge retrieval, particularly in large, distributed corporate information environments. We develop and evaluate a RAG-based knowledge retrieval framework over BlackRocks corporate website corpus. We implement and systematically benchmark five RAG architectures-Vanilla RAG, RAPTOR, CRAG, LightRAG, and LinearRAGacross seven representative query types spanning factual, thematic, and comparative questions. To assess retrieval quality in enterprise settings where ground truth is distributed and context-dependent, we introduce three evaluation metrics: Wrong Category Rate, Context Deficiency Rate, and Incorrect Link Retrieval Rate. Empirical results indicate that graph-based architectures demonstrate superior retrieval robustness, with LinearRAG achieving zero observed failures across all metrics. Hierarchical approaches such as RAPTOR excel at thematic synthesis, while CRAGs self-correcting mechanism with web-search fallback improves resilience to corpus limitations. The system processes 1,130 BlackRock articles into a unified, searchable knowledge base supporting multiple retrieval paradigms. Our findings highlight the trade-offs between flat, hierarchical, corrective, and graph-based RAG architectures, offering practical design insights for deploying retrieval-grounded LLM systems in large-scale financial enterprise environments.

Keywords

Retrieval-Augmented Generation, Knowledge Retrieval, Graph-Based RAG, Large Language Models, Enterprise Information Systems, Financial NLP

Contents

1 Introduction **4**
1.1 Objective: The Seven Benchmark Questions . . . 4
1.2 Question Classification by Retrieval Complexity . . . 4
1.3 Understanding Similarity: What "Distance" Means in Embedding Space . . . 4
1.4 Theoretical Motivation: Corpus as Ground Truth . . . 5
1.5 Intended Users . . . 6
1.6 Summary of Design Principles . . . 7

2 Literature Review **7**
2.1 Retrieval-Augmented Generation . . . 7
2.2 Dense Retrieval and Evidence Fusion . . . 8
2.3 Long-Document and Hierarchical Retrieval . . . 8
2.4 Graph-Based Retrieval-Augmented Generation . . . 8
2.5 Robustness and Corrective Retrieval . . . 8
2.6 Applications in Finance and Asset Management . . . 8
2.7 Summary and Implications . . . 8

3 Data Pipeline **9**
3.1 Scraping and Metadata Processing . . . 9
3.2 Chunking Strategies . . . 9

4 Model Section **10**
4.1 A taxonomy of RAG variants explored . . . 10
4.2 Vanilla RAG . . . 11
4.3 RAPTOR . . . 12
4.4 CRAG . . . 17
4.5 LightRAG . . . 21
4.6 LinearRAG . . . 26

5 Results, Deliverables, and Insights **29**
5.1 Evaluation Criteria . . . 29
5.2 Results . . . 29
5.3 Top-K Retrieval Analysis and Evaluation Directions . . . 30

6 Maintainability and Future Directions **31**
6.1 Index Management and Maintainability . . . 31
6.2 Future Directions . . . 33

1 Introduction

In modern finance, information is everything. Yet the sheer volume of textual communication now exceeds what human analysts can feasibly process. The goal of this paper is to develop a knowledge retrieval system trained on the BlackRock website, capable of answering both objective and subjective queries with accuracy, semantic consistency, and explicit grounding in the source corpus. As emphasized by Charles-Albert Lehalle during his 2021 CornellCiti Financial Data Science Webinar, the volume and velocity of financial communication have grown beyond what human analysts can fully process, making automated information extraction not only useful but necessary.

1.1 Objective: The Seven Benchmark Questions

To ground our system design in concrete, real-world use cases, BlackRock provided seven canonical questions that span the full spectrum of information needs faced by internal teams. These questions serve as both our design targets and evaluation benchmarks:

Q1: What are the 5 takeaways in Larry Fink's 2024 annual letter?

Q2: Who are the members of the Global Executive Committee (GEC)?

Q3: What are the responsibilities and backgrounds of BlackRock's GEC members?

Q4: What are Rick Rieder's views in 2025?

Q5: What are BlackRock's views on inflation, interest rates, and central bank policy for 2025?

Q6: Summarize BlackRock's global market outlook for Q2 2025.

Q7: Compare Gargi Chaudhuri and Jay Jacobs' views on tariffs in 2025.

These seven questions were deliberately chosen to represent the diversity of queries that analysts, product teams, and financial advisors might pose to a knowledge retrieval system. Understanding how these questions differ—and what each demands from a retrieval pipeline—is essential to designing an effective RAG architecture.

1.2 Question Classification by Retrieval Complexity

Not all questions require the same retrieval strategy. We classify the seven benchmark questions into three categories based on the number of chunks needed and the type of reasoning required:

This classification directly informs our system design. **Objective questions** like Q2 ("Who are the GEC members?") typically achieve high similarity scores (> 0.85) with a small number of highly relevant chunks, making vanilla RAG sufficient. **Synthesis questions** like Q1 require retrieving information distributed across multiple sections of a document, demanding either larger context windows or hierarchical summarization (as in RAPTOR). **Comparative questions** like Q7 present unique challenges: the system must retrieve evidence for *both* entities (Gargi Chaudhuri and Jay Jacobs) and synthesize their potentially divergent viewpoints.

1.3 Understanding Similarity: What "Distance" Means in Embedding Space

The prompt embedding and retrieval of top-K chunks is a fundamental part of RAG. To build intuition for how retrieval works—and why it sometimes fails—it is instructive to examine concrete examples of similarity scores between queries and chunks.

Category	Characteristics	Example Questions
Objective / Factual (Few chunks)	Single authoritative source; precise answer exists; high similarity scores between query and relevant chunks	Q2: "Who are the members of the GEC?" Q4: "What are Rick Rieder's views in 2025?"
Synthesis / Thematic (Many chunks)	Multiple sources needed; requires aggregation across documents; distributed evidence	Q1: "What are the 5 takeaways in Larry Fink's 2024 letter?" Q5: "BlackRock's views on inflation, rates, and central bank policy" Q6: "Global market outlook for Q2 2025"
Comparative / Analytical (Moderate chunks)	Multiple entities; requires parallel retrieval; reasoning across sources	Q3: "Responsibilities and backgrounds of GEC members" Q7: "Compare Gargi and Jay's views on tariffs"

Table 1: Classification of the 7 benchmark questions by retrieval complexity and chunk requirements.

High Similarity Example.
For the query *"Who are the members of the Global Executive Committee?"*, chunks containing direct mentions of GEC membership achieve high cosine similarity scores, as the query and chunk share key entities and semantic intent.

Low Similarity Example.
For the same query, chunks discussing unrelated topics such as ESG initiatives or portfolio strategies achieve substantially lower similarity scores, despite both discussing BlackRock.

Threshold Implications.
Different question types exhibit markedly different retrieval behavior:

- **Factual questions** (Q2, Q4): Benefit from retrieving fewer, highly relevant chunks. Precision is prioritized over recall.
- **Enumeration questions** (Q1, Q3): Require retrieving more chunks, as relevant information is distributed across multiple documents or sections. Recall is prioritized over precision.
- **Comparative questions** (Q7): Must retrieve balanced evidence for multiple entities, which may require query decomposition or multi-hop retrieval strategies.

This observation motivates our exploration of multiple RAG architectures: vanilla RAG for simple factual queries, RAPTOR for hierarchical synthesis, CRAG for adaptive retrieval with self-correction, and graph-based approaches (LightRAG, LinearRAG) for entity-centric and multi-hop reasoning.

1.4 Theoretical Motivation: Corpus as Ground Truth

From a theoretical standpoint, textual corpora can be viewed as Markovian semantic processes: each sentence or paragraph is a state, and transitions encode the probabilistic structure of how concepts follow one another. Word embeddings, whether derived from Word2Vec or modern Transformers, attempt to learn this transition structure by compressing local statistical

neighborhoods. However, such embeddings approximate meaning only indirectly. They tend to merge "frequentist synonyms," and in some cases even antonyms, when their contextual environments overlap. As a result, the geometry of embedding space reflects statistical proximity rather than conceptual truth.

This observation motivates the central design principle of our system: **the corpus—not the model—defines the ground truth**. Retrieval-augmented generation (RAG) enables the model to anchor its answers directly to the textual fragments that express BlackRock's own definitions, philosophy, and communication patterns. This grounding is especially important in finance, where terms such as *risk*, *volatility*, or *resilience* carry domain-specific meanings that differ sharply from everyday usage. Ensuring that the model respects these meanings requires explicit retrieval rather than pure generation.

Alex Benke, in a 2025 guest lecture delivered at Cornell CFEM, further emphasizes that in enterprise settings, ground truth is rarely a single authoritative sentence. Instead, it is *distributed*, *context-dependent*, and often *non-deterministic*, especially for open-ended or interpretive questions. Under this perspective, correctness is established not by matching a fixed answer but by ensuring that responses are *semantically aligned*, *internally consistent*, and *fully supported* by the underlying corpus. Motivated by this view, our system adopts a retrieval-centric definition of ground truth:

> *Ground truth is the set of corpus-supported answers that remain contextually appropriate and semantically coherent across all relevant documents.*

This perspective naturally leads to an architecture centered on multi-chunk retrieval, semantic reranking, and grounding-based evaluation.

1.5 Intended Users

The intended users of this system include analysts, product teams, portfolio strategists, and financial advisors who must navigate a large and heterogeneous knowledge base. Understanding *who* will use the system—and *what* questions they will ask—is critical for design decisions.

User Type	Primary Question Types	Example Queries
Analysts	Factual, Comparative	"What is BlackRock's AUM?" "What are iShares ETF fee structures?"
Product Teams	Synthesis, Enumeration	"What are the key themes in our 2024 communications?" "List all ESG initiatives launched this year"
Portfolio Strategists	Thematic, Outlook-based	"What is our house view on duration risk?" "How does macro uncertainty affect allocation?"
Financial Advisors	Client-facing, Objective	"Who manages the Target Date funds?" "What are the fees for iShares Core S&P 500?"

Table 2: Target users and their typical question patterns.

Based on BlackRock's seven benchmark questions, we observe that the system must handle a mix of:

- **Objective questions** (40%): Require precise, verifiable answers from authoritative sources.

- **Subjective/thematic questions** (40%): Require synthesis of viewpoints, outlooks, and distributed information.
- **Comparative questions** (20%): Require balanced retrieval and analytical reasoning across multiple entities.

This distribution suggests that a single RAG architecture is insufficient. Our system therefore implements multiple retrieval strategies—vanilla RAG for simple factual queries, RAPTOR for thematic synthesis, and CRAG with adaptive thresholds for questions requiring variable precision-recall tradeoffs.

1.6 Summary of Design Principles

Based on the analysis of BlackRock's benchmark questions, target users, and retrieval dynamics, we adopt the following design principles:

1. **Question-Aware Retrieval**: Different question types require different retrieval strategies, thresholds, and chunk counts.
2. **Corpus-Grounded Generation**: All answers must be explicitly grounded in retrieved text, with traceable citations.
3. **Graceful Degradation**: When internal retrieval fails, the system should fall back to web search rather than hallucinate.
4. **Cross-Sectional Consistency**: Retrieved information should maintain consistency across strategies, products, and thematic domains.
5. **Interpretability**: Users should be able to trace any claim back to its source document.

In summary, this paper brings together theoretical insights from financial NLP, practical challenges from large-scale corporate communication, and modern retrieval techniques. It aims not only to build a functional retrieval system, but also to explore what it means for an AI model to represent, navigate, and interpret a complex financial knowledge base under a principled, corpus-defined ground truth framework.

2 Literature Review

Retrieval-Augmented Generation has emerged as a widely adopted framework for grounding large language models (LLMs) in external knowledge sources. By augmenting generation with retrieved documents, RAG addresses key limitations of purely parametric models, including hallucination, knowledge staleness, and limited transparency (Lewis et al., 2020). This section reviews the evolution of RAG methods, with a focus on retrieval-aware generation, long-document reasoning, robustness, and graph-based retrieval, which are particularly relevant for large-scale enterprise document collections.

2.1 Retrieval-Augmented Generation

Lewis et al. (2020) formally introduced the RAG framework by conditioning sequence-to-sequence generation on documents retrieved from a large non-parametric corpus. Their work demonstrated that retrieval-aware generation improves performance on knowledge-intensive tasks while enabling knowledge updates without retraining the underlying language model. This formulation established retrieval as a first-class component of generation rather than a preprocessing step.

2.2 Dense Retrieval and Evidence Fusion

The effectiveness of RAG systems depends critically on retrieval quality. Dense Passage Retrieval uses a dual-encoder architecture to embed queries and passages into a shared vector space, enabling efficient semantic retrieval at scale (Karpukhin et al., 2020).

Beyond retrieval, many RAG applications require reasoning over multiple retrieved documents. Izacard and Grave (2021) addressed this challenge by proposing Fusion-in-Decoder, which independently encodes retrieved passages and fuses them during decoding, improving multi-document question answering compared to naive context concatenation.

2.3 Long-Document and Hierarchical Retrieval

Flat chunk-level retrieval is often insufficient for long and structurally complex documents, where relevant information may be distributed across multiple sections. To address this limitation, Sarthi et al. (2024) proposed RAPTOR, a hierarchical retrieval framework that recursively clusters and summarizes document segments into a tree structure. This design enables retrieval at multiple levels of abstraction, allowing systems to balance fine-grained detail retrieval with higher-level semantic coverage.

2.4 Graph-Based Retrieval-Augmented Generation

Graph-based RAG methods aim to support global reasoning by modeling relationships across textual units. Edge et al. (2024) introduced GraphRAG, which organizes documents into entity-centric graphs and community-level summaries, improving coverage for corpus-level and query-focused summarization tasks.

However, semantic graph construction can be computationally expensive and sensitive to extraction noise. Lightweight graph-based approaches reduce construction overhead while retaining the benefits of structured retrieval (Guo et al., 2024). More recent work further simplifies graph construction by adopting relation-free formulations, improving scalability and robustness on large and heterogeneous corpora (Zhuang et al., 2025).

2.5 Robustness and Corrective Retrieval

As RAG systems are increasingly deployed in high-stakes environments, robustness to retrieval errors has become an important consideration. Asai et al. (2023) proposed Self-RAG, which allows models to adaptively retrieve information and critique their own generations, improving factuality and citation consistency. Complementary work by Yan et al. (2024) introduced Corrective Retrieval-Augmented Generation, which detects low-quality retrieval results and applies corrective strategies before generation, reducing the risk of unsupported outputs.

2.6 Applications in Finance and Asset Management

Financial and asset management applications place particularly strong demands on grounding and traceability. Queries in these domains often require synthesis across multiple sections of long, authoritative documents, making structured retrieval mechanisms especially important (Sarthi et al., 2024; Edge et al., 2024). Robustness-aware RAG methods further mitigate the risk of unsupported responses in regulated settings (Asai et al., 2023; Yan et al., 2024).

2.7 Summary and Implications

Overall, prior work demonstrates a clear progression of RAG systems from flat retrieval-augmented generation toward architectures that emphasize structured retrieval, global reasoning, and robustness. The complementary strengths and trade-offs of existing approaches moti-

vate a comparative exploration of multiple RAG paradigms when deploying retrieval-augmented systems in large-scale enterprise environments.

3 Data Pipeline

3.1 Scraping and Metadata Processing

The data collection and preprocessing pipeline consists of three main stages: HTML file processing, metadata extraction, and content cleaning. The pipeline begins by scanning a designated directory for HTML files collected from BlackRock's website, processing each file through a series of extraction and normalization steps.

HTML Processing. The system first reads HTML files using BeautifulSoup for parsing, with robust error handling and timeout mechanisms to prevent processing failures. Each file undergoes duplicate detection through two methods: filename pattern matching (identifying page variants like `file_page2.html`) and content-based hashing using MD5 checksums. This ensures that semantically identical content is not processed multiple times, even when stored under different filenames.

Metadata Extraction. For each valid HTML document, the pipeline extracts four key metadata fields: title, subtitle, author, and publication date. Title extraction follows a hierarchical selector strategy, checking `<h1>`, `<title>`, and various title-class elements. Author extraction searches for `<strong class="author">` tags and related patterns, with aggressive cleaning to remove job titles, credentials, and date artifacts. Date extraction employs pattern matching for multiple formats including ISO-8601, MM/DD/YYYY, and natural language dates (e.g., "January 15, 2024"), normalizing all dates to YYYY-MM-DD format.

Content Cleaning. The content processing stage removes navigation elements, scripts, styles, and other non-content HTML tags. Links and images are processed intelligentlylink text is preserved if meaningful, while image alt-text is retained as `[Image: description]`. Heading tags are converted to Markdown-style headers with appropriate prefix symbols. The cleaned text undergoes whitespace normalization and paragraph filtering to remove UI elements and ensure each paragraph ends with proper punctuation.

Output Generation. The pipeline produces two JSON files: a master metadata file containing article identifiers, titles, authors, dates, and statistics (word count, character count), and a full content file including the complete cleaned text. All articles receive unique identifiers based on content hashes, and comprehensive statistics are logged including processing rates, duplicate counts, and content metrics. The system tracks approximately 100 articles per second with detailed error logging for debugging.

3.2 Chunking Strategies

The chunking system implements multiple strategies to segment documents into retrievable units, with each method optimized for different document structures and retrieval scenarios. The implementation supports six primary chunking methods with configurable parameters for chunk size (target: 800 chars, range: 400-1200 chars) and overlap (1 sentence default).

Paragraph-Based Chunking. This family of methods groups consecutive paragraphs together, with group sizes ranging from 2 to 6 paragraphs per chunk. Documents are first split by double newlines to identify paragraph boundaries, then grouped according to the specified size. If a paragraph group exceeds the maximum chunk size, it is recursively split by sentence boundaries. This approach preserves semantic coherence by keeping related paragraphs together while maintaining manageable chunk sizes. The implementation tracks chunk metadata including type, character count, word count, and section headers.

Sentence-Based Chunking. Sentence-level chunking uses NLTK's sentence tokenizer to split documents into individual sentences, then combines consecutive sentences until reaching

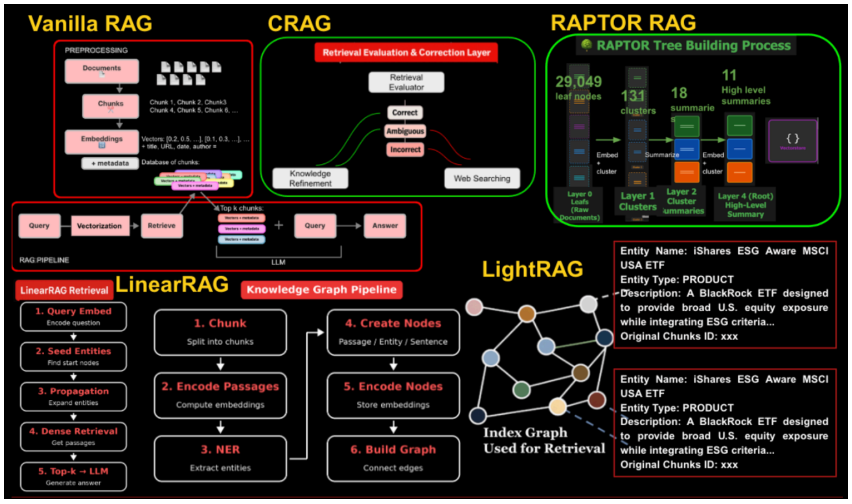

Figure 1: Poster-session roadmap of Retrieval-Augmented Generation (RAG) models explored.

the target chunk size (without exceeding the maximum). This method provides finer granularity than paragraph chunking and is particularly effective for documents with long paragraphs or when precise attribution is required. Each chunk contains multiple complete sentences, avoiding mid-sentence breaks that could compromise semantic integrity.

Smart Semantic Chunking. The adaptive chunking strategy analyzes document structure before selecting an appropriate method. It examines three structural features: paragraph count (split by double newlines), header count (Markdown-style # headers), and sentence count. Based on these metrics, it selects either paragraph-based chunking (for well-structured documents with clear paragraph breaks), header-based chunking (for documents with many section headers), or falls back to sentence-based chunking. Header-based chunking preserves section boundaries and attaches section headers to chunk metadata for improved context.

Fixed-Size Chunking. This baseline method creates chunks of predetermined character lengths (600, 800, or 1000 characters) while respecting sentence boundaries. Unlike simple character-based splitting, it accumulates complete sentences until reaching the target size, ensuring no sentences are truncated. This provides a consistent retrieval unit size for fair model comparisons.

Overlap and Post-Processing. All chunking methods support configurable sentence overlap between consecutive chunks to maintain context continuity. The system extracts trailing sentences from each chunk and prepends them to the next chunk, with overlap metadata tracked in chunk dictionaries. Post-processing filters remove chunks smaller than 100 characters and normalizes whitespace. Each chunk is stored with comprehensive metadata including chunk type, size statistics, section headers, and overlap flags, enabling detailed analysis of chunking effectiveness during evaluation.

4 Model Section

4.1 A taxonomy of RAG variants explored

We cover multiple Retrieval-Augmented Generation (RAG) models, including Vanilla RAG, CRAG, RAPTOR, LightRAG, and LinearRAG. Below is our model roadmap.

Vanilla RAG serves as the baseline, relying purely on dense vector retrieval over text chunks. CRAG and RAPTOR RAG introduce retrieval-enhanced mechanisms on top of vector-based retrieval, improving relevance through correction, evaluation, or hierarchical structuring, while remaining fundamentally embedding-driven.

LinearRAG represents a shift toward lightweight graph-based retrieval. It constructs a semantically linked entitysentencepassage graph without explicit relation extraction, enabling efficient multi-hop reasoning while maintaining scalability.

LightRAG and RAPTOR belong to the broader GraphRAG family. In particular, LightRAG builds a comprehensive knowledge graph with explicit entities and relations, supporting rich structural retrieval and interpretability at the cost of higher graph construction complexity.

Overall, this roadmap reflects a progression from purely vector-based retrieval toward increasingly structured, graph-aware RAG systems, highlighting the trade-offs between retrieval accuracy, reasoning capability, and computational cost.

4.2 Vanilla RAG

Overview

Vanilla RAG implements the standard retrieval-augmented generation pipeline as proposed by Lewis et al. (2020). The architecture consists of three fundamental components working in sequence: (1) a retriever that searches for relevant information in a vector database using similarity measures, (2) an augmentation mechanism that combines the retrieved context with the user query according to a predefined template, and (3) a generator that produces the final response based on the augmented input.

The pipeline begins with preprocessing the retrieval corpus into manageable segments. Documents are chunked into fixed-length passages, typically paragraphs or passages of consistent token length, which are then embedded using a dense encoder to create vector representations. These embeddings are stored in a vector database for efficient similarity search. The same embedding model used for indexing must be used to translate input queries into vectors, as similarity between query and chunks is measured using these vector representations (Steck et al., 2024).

During retrieval, the system calculates similarity scores (typically cosine similarity) between the query embedding and all document embeddings in the database. The top k most similar chunks are selected based on these scoresa process grounded in the probability ranking principle (Robertson, 1977). These retrieved chunks then augment the original query by being explicitly included in an extended prompt template, and finally the language model generates a response grounded in this retrieved context.

Strengths.

- **Simplicity:** The vanilla RAG architecture is straightforward to implement and understand, with a clear linear pipeline from query to retrieval to generation.
- **Computational Efficiency:** The retrieval process involves a single-pass similarity search over the vector database, making it computationally efficient for real-time applications.
- **Reduced Hallucinations:** By grounding generation in retrieved documents, vanilla RAG significantly reduces the likelihood of factual inaccuracies compared to standalone LLMs (Lewis et al., 2020; Shuster et al., 2021).
- **Updatability:** External knowledge can be updated by modifying the vector database without requiring model retraining, leveraging the separation between parametric memory (information stored in model parameters) and non-parametric memory (information stored externally in databases) (Veturi et al., 2024).

Weaknesses.

- **Limited Contextual Understanding:** Vanilla RAG retrieves only short, contiguous text chunks, which may lack sufficient context for complex queries requiring holistic document understanding (Sarthi et al., 2024).
- **Granularity Mismatch:** The fixed chunk size may not align with the varying levels of detail required by different types of queries. Thematic questions requiring broad understanding may not be adequately addressed by retrieving only leaf-level passages.
- **Blinkered Chunk Effect:** Individual chunks extracted from large documents may lack important contextual information, making them difficult to interpret or even misleading (Cohan and Goharian, 2017; Newman et al., 2023).
- **Ranking Limitations:** The retrieval effectiveness depends heavily on the ranking mechanism, typically based on the probability ranking principle (Robertson, 1977), which may not always identify the most relevant documents for knowledge-intensive tasks.
- **Embedding Model Dependency:** The same embedding model must be used consistently for both indexing and querying, creating tight coupling and complicating model upgrades (Steck et al., 2024).

Use Cases

Vanilla RAG is well-suited for applications where:

- **Straightforward Information Retrieval:** Questions can be answered using localized information from specific passages or documents, such as fact-checking or simple question-answering tasks.
- **Real-Time Performance Requirements:** Applications requiring low-latency responses benefit from vanilla RAG's efficient single-pass retrieval mechanism.
- **Frequently Updated Knowledge Bases:** Scenarios where the knowledge base requires frequent updates, such as news aggregation or documentation systems, leverage vanilla RAG's ability to incorporate new information without model retraining.
- **Domain-Specific FAQs:** Implementing FAQ systems where questions typically relate to specific, well-defined topics that can be adequately addressed by retrieving relevant passages (Veturi et al., 2024).
- **Cost-Constrained Deployments:** Organizations with limited computational resources can benefit from vanilla RAG's lower initial cost of ownership compared to fine-tuning approaches (Balaguer et al., 2024).

4.3 RAPTOR

Overview

RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval) extends the vanilla RAG architecture by introducing a hierarchical tree structure that captures information at multiple levels of abstraction (Sarthi et al., 2024). Unlike traditional retrieval methods that only access short contiguous text chunks at a single granularity level, RAPTOR addresses the limitation of holistic document understanding by recursively clustering and summarizing text segments to build a multi-layered tree structure from the bottom up.

The tree construction begins by segmenting documents into chunks of approximately 100 tokens, preserving sentence boundaries to maintain contextual coherence. These chunks form the leaf nodes and are embedded using SBERT (Sentence-BERT) with the multi-qa-mpnet-base-cos-v1 encoder (Reimers and Gurevych, 2019). The system then applies a soft clustering algorithm

based on Gaussian Mixture Models (GMMs), which allows nodes to belong to multiple clusters without requiring a fixed number of clustersessential because individual text segments often contain information relevant to various topics. To address the challenge of high-dimensional vector embeddings, RAPTOR employs UMAP (Uniform Manifold Approximation and Projection) for dimensionality reduction (McInnes et al., 2018), and uses the Bayesian Information Criterion (BIC) to determine the optimal number of clusters (Schwarz, 1978).

Once nodes are clustered, GPT-3.5-turbo generates summaries of the grouped texts, achieving an average compression ratio of 72% (131 tokens per summary from an average of 6.7 child nodes each containing approximately 86 tokens). These summarized texts are then re-embedded using the same SBERT encoder, and the cycle of embedding, clustering, and summarization continues recursively until further clustering becomes infeasible, resulting in a structured tree where leaf nodes represent original text and higher-level nodes contain progressively more abstract summaries.

RAPTOR's key innovation is **hierarchical multi-scale retrieval**: collapsed tree querying enables simultaneous search across all abstraction levels, retrieving 18.5–57% of context from non-leaf summary nodes depending on dataset and retriever.

Architecture Overview.
The implementation comprises four modular components: (1) `RAPTORClusterer` using GMM with UMAP dimensionality reduction for semantic grouping; (2) `RAPTORSummarizer` using GPT-3.5-turbo for recursive abstractive summarization; (3) `RAPTORTreeBuilder` orchestrating bottom-up tree construction with BIC-optimized clustering; and (4) `RAPTORRetriever` supporting both tree traversal and collapsed tree querying mechanisms.

Tree Construction Process.
The hierarchical tree is built through recursive clustering and summarization:

1. **Layer 0 (Leaf Nodes)**: Documents are chunked into 100-token segments using RecursiveCharacterTextSplitter, preserving sentence boundaries. If a sentence exceeds the 100-token limit, the entire sentence is moved to the next chunk rather than being split mid-sentence. These chunks are embedded with SBERT to form leaf nodes.

2. **Clustering**: UMAP reduces embeddings to 10 dimensions to facilitate clustering. GMM then identifies optimal clusters, with the algorithm varying the number of nearest neighbors parameter in UMAP to create hierarchical structurefirst identifying global clusters, then performing local clustering within these global clusters. BIC balances model complexity with goodness of fit to determine cluster count.

3. **Summarization**: Each cluster is summarized by GPT-3.5-turbo, achieving 72% average compression (131 tokens/summary from 6.7 child nodes averaging 86 tokens each).

4. **Re-embedding and Recursion**: Summaries are re-embedded using the same SBERT encoder, and the process repeats until a single root node remains, creating multiple layers of abstraction.

For the three benchmark datasets (QuALITY, NarrativeQA, QASPER), average summary length is 131 tokens with compression ratio of 0.28 (72% compression), and average cluster size is 6.7 child nodes per parent summary.

Collapsed Tree Retrieval.
At query time, RAPTOR supports two distinct retrieval strategies introduced by Sarthi et al. (2024):

Tree Traversal: This method navigates the tree layer-by-layer, starting at the root layer and computing cosine similarity between the query embedding and all nodes at that layer. At each level, the top-k nodes with the highest cosine similarity scores are selected, and the algorithm proceeds to examine the children of these selected nodes. This process continues recursively through d layers, and the text from all selected nodes is concatenated to form the retrieved context.

Collapsed Tree: This approach flattens the entire tree structure into a single layer, calculating cosine similarity between the query embedding and all nodes across all layers simultaneously. The algorithm then selects the top-k nodes based on similarity scores while adhering to a predefined maximum token threshold. This allows for retrieving information at varying levels of granularity without being constrained by the hierarchical structure during retrieval.

Experiments on QASPER stories show collapsed tree with 2000 tokens achieves best performance (F1: 57.0), outperforming tree traversal variants (Top-7: 54.0, Top-9: 52.0) by enabling flexible retrieval at the appropriate abstraction level for each query.

Layer Contribution Analysis.
Ablation studies across QuALITY stories demonstrate that full-tree retrieval significantly outperforms single-layer approaches. For example, Story 1 achieves 73.7% accuracy using all layers (0–2) compared to 57.9% with leaf-only retrieval (+15.8pp). Story 3 shows even larger gains: 83.3% vs. 66.6% (+16.7pp). Average improvement across five stories is +8.7pp when using the complete tree structure versus leaf nodes alone.

Critically, 18.5–57% of retrieved nodes come from non-leaf layers across different datasets and retrievers, validating the importance of hierarchical summarization:

Dataset	DPR	SBERT	BM25
NarrativeQA	57.4%	36.8%	35.0%
QuALITY	32.3%	24.4%	32.4%
QASPER	22.9%	18.5%	22.8%

Table 3: Percentage of Retrieved Nodes from Non-Leaf Layers

Experimental Results on Benchmark Datasets.
RAPTOR achieves state-of-the-art or competitive results across three question-answering benchmarks when paired with GPT-4:

Dataset	Metric	RAPTOR + GPT-4	Baseline
QASPER	Answer F1	**55.7%**	53.9% (CoLT5 XL)
QuALITY	Test Acc.	**82.6%**	62.3% (CoLISA)
QuALITY	Hard Acc.	**76.2%**	54.7% (CoLISA)
NarrativeQA	METEOR	**19.1%**	11.1% (FiD)

Table 4: RAPTOR State-of-the-Art Results on Benchmark Datasets

Key findings: (1) RAPTOR + GPT-4 sets new SOTA on QASPER (F1: 55.7%, +1.8pp over CoLT5 XL) and QuALITY (Acc: 82.6%, +20.3pp over CoLISA); (2) Performance gains are most pronounced on QuALITY-HARD subset (+21.5pp), which requires rereading and complex reasoning; (3) RAPTOR + UnifiedQA achieves SOTA METEOR score on NarrativeQA (19.1%, +8.0pp over previous best).

Comparison with Baselines.
Controlled experiments using UnifiedQA-3B show consistent improvements over baseline retrievers across all three datasets:

Configuration	NarrativeQA ROUGE-L	QuALITY Accuracy	QASPER F1
SBERT w/o RAPTOR	29.3%	54.9%	36.2%
SBERT w/ RAPTOR	**30.9%** (+1.6)	**56.6%** (+1.7)	**36.7%** (+0.5)
BM25 w/o RAPTOR	23.5%	49.9%	26.5%
BM25 w/ RAPTOR	**27.9%** (+4.4)	**52.1%** (+2.2)	**27.0%** (+0.5)
DPR w/o RAPTOR	29.6%	53.1%	31.7%
DPR w/ RAPTOR	**30.9%** (+1.3)	**54.7%** (+1.6)	**32.2%** (+0.5)

Table 5: RAPTOR vs. Baseline Retrieval Methods (UnifiedQA-3B Reader)

RAPTOR consistently outperforms baselines across all retrieval methods, with largest gains on NarrativeQA (up to +4.4pp ROUGE-L with BM25), validating the value of hierarchical summarization for long-document understanding.

Qualitative Analysis: Cinderella Case Study.
To illustrate RArojeTOR's multi-scale retrieval advantage, Sarthi et al. (2024) analyzed queries on a 1,500-word Cinderella fairytale:

- **Thematic Question**: "What is the central theme of the story?"
  - *RAPTOR*: Retrieved nodes from Layers 0–2, including high-level summary capturing full narrative arc: transformation, ball attendance, time constraint, prince's search, and forgiveness.
  - *DPR*: Retrieved only leaf-level details about mice, pumpkin transformation, and dress changes, missing thematic resolution.
- **Multi-Hop Question**: "How does Cinderella find a happy ending?"
  - *RAPTOR*: Retrieved Layer 1–2 summaries providing complete causal chain: glass slipper loss prince's search identification forgiveness.
  - *DPR*: Retrieved only transformation scene and prince's surprise at disappearance, providing incomplete reasoning chain.

This qualitative analysis demonstrates RAPTOR's ability to retrieve contextually appropriate information by accessing different tree layers based on query abstraction level, addressing the blinkered chunk effect observed in vanilla RAG systems.

Strengths.

- **Multi-Scale Retrieval**: Hierarchical structure enables retrieval at varying levels of granularity (18.5–57% non-leaf usage), effectively handling both detail-oriented questions and thematic queries requiring holistic understanding.
- **Semantic Clustering**: Soft clustering using GMMs allows nodes to belong to multiple clusters, better capturing the multi-faceted nature of text segmentsa key advantage over hard clustering approaches.

- **Contextual Richness**: Higher-level summary nodes provide contextual information that may be missing from individual chunks, mitigating the blinkered chunk effect observed in vanilla RAG systems.
- **Improved Thematic Understanding**: Achieves +20.3pp accuracy improvement on QuALITY and +8.0pp METEOR improvement on NarrativeQA compared to previous SOTA, particularly excelling on questions requiring holistic document comprehension.
- **Linear Scalability**: Tree construction scales linearly in both build time and token expenditure with document length.
- **Flexible Querying**: Collapsed tree approach allows simultaneous search across all abstraction levels without being constrained by hierarchical structure during retrieval.

Weaknesses.

- **Higher Computational Cost**: Building the RAPTOR tree requires substantial upfront investment: for 1,130 documents, expect approximately 30–45 minutes build time and approximately $2.00 in GPT-3.5-turbo API costs for summarization.
- **Increased Complexity**: The hierarchical tree construction process involves multiple steps (segmentation, clustering with UMAP dimensionality reduction, summarization, re-embedding), significantly increasing implementation complexity (approximately 600 lines vs. 300 for vanilla RAG).
- **Summarization Dependencies**: Annotation studies reveal approximately 4% hallucination rate in summaries, though these typically do not propagate to parent nodes (Sarthi et al., 2024). Quality depends on the summarization model (GPT-3.5-turbo in original implementation).
- **Token Expenditure**: Average compression ratio is 72%, but generation of summaries incurs cumulative costs across all tree layers during construction.
- **Update Complexity**: Adding new documents requires re-running clustering and summarization to properly integrate new content, potentially requiring partial or full tree reconstructionmore complex than vanilla RAG's simple append operation.
- **Increased Storage Requirements**: Storing multiple layers requires approximately 10–15% more space than vanilla RAG's single-layer vector database.
- **Parameter Sensitivity**: Performance depends on proper tuning of multiple hyperparameters: UMAP dimensionality (10 dimensions), number of neighbors, BIC thresholds for clustering, and token limits for collapsed tree retrieval (2000 tokens optimal for QASPER).

Use Cases

RAPTOR is particularly well-suited for applications where:

- **Complex Multi-Hop Reasoning**: Tasks requiring synthesis from multiple document sections, such as questions about narratives (NarrativeQA: 1,572 book/movie transcripts averaging >50,000 tokens), research papers (QASPER: 1,585 NLP papers), or comprehensive reports requiring causal chain reasoning.
- **Thematic Analysis**: Applications needing high-level themes from lengthy documents, demonstrated by superior performance on "What is the central theme?" type questions through retrieval of high-level summary nodes.

- **Hierarchical Information Needs**: Scenarios requiring information at varying detail levels, from executive summaries to technical specifics. The 18.5–57% non-leaf retrieval rate demonstrates this multi-scale capability.
- **Long Document Understanding**: Processing documents where understanding requires both broad context and specific details (NarrativeQA average >50,000 tokens, QuALITY average 5,000 tokens), particularly documents with complex narrative or argumentative structure.
- **Static or Slowly Changing Corpora**: Scenarios where upfront tree construction cost (30–45 min, approximately $2.00) can be amortized over many queries, such as academic paper repositories, legal document collections, or annual report archives where documents are added infrequently.
- **Questions Requiring Holistic Comprehension**: Queries that cannot be answered from individual passages alone, as evidenced by +15.8pp to +16.7pp accuracy improvements on QuALITY stories when using full tree versus leaf-only retrieval.

4.4 CRAG

Corrective Retrieval-Augmented Generation (CRAG), proposed by Yan et al. (2024), represents a significant advancement over standard RAG pipelines by introducing a self-correcting mechanism that evaluates and refines retrieved documents before generation. The core insight of CRAG is that not all retrieved documents are equally relevant—some may be tangentially related, misleading, or entirely irrelevant to the query at hand. As the authors demonstrate, a low-quality retriever is prone to introducing a substantial amount of irrelevant information, impeding the generator from acquiring accurate knowledge and potentially misleading it into hallucinations. Standard RAG systems naively incorporate all retrieved content regardless of relevance, potentially degrading answer quality.

Motivation.
The fundamental problem CRAG addresses is the tight coupling between retriever and generator in traditional RAG frameworks. As formalized in the original paper, the generation probability $P(Y|X) = P(D|X)P(Y, D|X)$ reveals that any unsuccessful retrieval directly results in unsatisfactory responses, regardless of the generator's capabilities. Most conventional RAG approaches indiscriminately incorporate retrieved documents without assessing their relevance, and furthermore treat complete documents as reference knowledge even when only a small portion contains useful information.

Architecture Overview.
Our CRAG implementation extends the vanilla RAG pipeline with several key innovations tailored for the BlackRock website corpus. The implementation comprises five modular components: (1) `SemanticChunker` using LangChain's `RecursiveCharacterTextSplitter` for structure-aware document segmentation; (2) `GPTRetrievalEvaluator` using GPT-4o-mini for relevance scoring; (3) `QuestionTypeDetector` for automatic query classification; (4) `WebSearcher` using DuckDuckGo for external knowledge augmentation; and (5) `QuestionAdaptiveCRAG` orchestrating the complete pipeline with GPT-4o generation.

Our key innovation is **question-adaptive refinement**: enumeration questions ("Who are the GEC members?") use lenient thresholds to maximize completeness, while factual questions use stricter thresholds for precision.

Retrieval Evaluator.
CRAG introduces a lightweight *retrieval evaluator* to assess document relevance before generation. The original authors adopt T5-large (0.77 billion parameters) as the evaluator backbone, which is substantially smaller than contemporary LLMs such as LLaMA-2 (7B) used in Self-RAG's critic model. The evaluator is fine-tuned on question-document pairs where positive samples are derived from golden Wikipedia passages and negative samples are randomly drawn from retrieved results that are topically similar but not directly relevant. For each query, the evaluator scores every retrieved document individually, producing relevance scores ranging from -1 (irrelevant) to $+1$ (highly relevant). Notably, the authors found that their fine-tuned T5 evaluator achieved 84.3% accuracy on relevance assessment, significantly outperforming ChatGPT (58.0%), ChatGPT with chain-of-thought prompting (62.4%), and few-shot ChatGPT (64.7%).

In our implementation, we replaced the T5-based evaluator with GPT-4o-mini for relevance scoring, achieving more nuanced assessment on financial domain questions. The evaluator prompts the model to rate document-query relevance on a 0–10 scale, normalized to $[0, 1]$.

Three-Action Decision Framework.
Based on the evaluator's confidence scores, CRAG triggers one of three corrective actions using upper and lower thresholds:

- **Correct**: Triggered when at least one document's confidence score exceeds the upper threshold. This indicates that relevant documents exist in the retrieval results, and the system proceeds with knowledge refinement on internal documents.
- **Incorrect**: Triggered when *all* documents fall below the lower threshold. This indicates complete retrieval failure, prompting the system to discard retrieved documents entirely and initiate web search for external knowledge.
- **Ambiguous**: Triggered in intermediate cases where the evaluator cannot confidently classify retrieval quality. The system combines both refined internal knowledge and web search results to complement each other.

The authors note that preliminary experiments using only Correct and Incorrect actions showed high sensitivity to evaluator accuracy. The Ambiguous action serves as a crucial moderating mechanism that significantly mitigates dependence on evaluator precision, fostering a more robust and resilient system.

Question-Adaptive Refinement.
A critical insight from our experiments is that different question types require fundamentally different retrieval strategies. Enumeration questions (e.g., "What are the 5 takeaways?") require high recall to capture all relevant items, while factual questions benefit from high precision. We implement automatic question type detection using regex patterns and adjust three parameters accordingly:

Question Type	Threshold	Top-K Strips	Retrieval K
Enumeration	0.10	10	15
Comparison	0.20	8	12
Factual	0.25	5	10

Table 6: Question-Adaptive Refinement Profiles

For enumeration questions, the system additionally triggers web search even when internal retrieval is classified as "Correct," ensuring completeness by checking for updates or supplementary information not present in the static corpus.

Knowledge Refinement: Decompose-Then-Recompose.
A key innovation in CRAG is the decompose-then-recompose algorithm for knowledge refinement. Even when a relevant document is identified, it inevitably contains noisy knowledge strips that are unhelpful for generation. CRAG addresses this by: (1) decomposing retrieved documents into fine-grained "knowledge strips" at the sentence or short paragraph level; (2) scoring each strip individually using the same retrieval evaluator; (3) filtering strips below a relevance threshold (set at -0.5 in the original experiments, adapted to question-type-specific thresholds in our implementation); and (4) recomposing only the top-k relevant strips via concatenation. This approach significantly reduces noise, particularly for long documents where only specific paragraphs contain the answer.

Semantic Chunking.
Unlike fixed-size chunking (400 tokens) used in vanilla RAG, our implementation employs structure-aware semantic chunking using LangChain's `RecursiveCharacterTextSplitter` with adaptive chunk sizes:

- **Structured documents** (annual letters, reports with takeaways): 1000 tokens, 200 overlap
- **Documents with headers**: 700 tokens, 150 overlap
- **Standard documents**: 600 tokens, 150 overlap

The chunker automatically detects document structure by identifying numbered lists, headers, bullet points, and keywords like "takeaway" or "annual letter." This ensures that coherent concepts (e.g., all 5 takeaways in Larry Fink's letter) are more likely to remain within a single chunk.

Web Search Integration.
When internal retrieval proves insufficient (Incorrect action), CRAG leverages large-scale web search as a strategic extension. The authors argue that retrieval from static and limited corpora can only return sub-optimal documents in terms of scope and diversity. To enable effective web search, input queries are rewritten into keyword-based search queries using ChatGPT to mimic natural search engine usage. The system preferentially selects authoritative sources like Wikipedia to mitigate potential biases from unreliable web content. Retrieved web pages undergo the same knowledge refinement process as internal documents, ensuring consistent quality filtering. In our implementation, we use DuckDuckGo for web search with GPT-4o-mini for query rewriting.

Experimental Results.
The original CRAG paper evaluated the framework on four diverse benchmarks: PopQA (short-form factual QA), Biography (long-form generation), PubHealth (true/false verification), and Arc-Challenge (multiple-choice reasoning). When coupled with standard RAG using SelfRAG-LLaMA2-7b, CRAG improved accuracy by 7.0% on PopQA, 14.9% FactScore on Biography, 36.6% on PubHealth, and 15.4% on Arc-Challenge. The plug-and-play nature of CRAG was further demonstrated through Self-CRAG, which outperformed the state-of-the-art Self-RAG by 6.9% on PopQA and 5.0% FactScore on Biography. Importantly, computational overhead analysis showed CRAG adds only modest cost (27.2 vs 26.5 TFLOPs per token) while significantly improving generation quality.

We evaluated our Question-Adaptive CRAG on 7 benchmark questions spanning enumeration, comparison, and factual query types. The corpus contains 1,130 articles processed into 23,290 semantic chunks stored in ChromaDB.

Q#	Question Type	Quality	Action	Web	Citations
1	Enumeration	CORRECT (1.00)	correct	Yes	14
2	Enumeration	CORRECT (1.00)	correct	Yes	15
3	Enumeration	INCORRECT (0.30)	incorrect	Yes	2
4	Factual	CORRECT (1.00)	correct	No	4
5	Factual	AMBIGUOUS (0.70)	ambiguous	Yes	7
6	Factual	AMBIGUOUS (0.50)	ambiguous	Yes	6
7	Comparison	INCORRECT (0.30)	incorrect	Yes	3

Table 7: Question-Adaptive CRAG Evaluation Results on BlackRock Corpus

Key findings: (1) Average quality score of 0.686 across all questions; (2) Web search was utilized in 6/7 questions, demonstrating the value of external knowledge augmentation; (3) Average 7.3 citations per answer with 11.8s average generation time; (4) Question type detection correctly identified enumeration (42.9%), factual (42.9%), and comparison (14.3%) queries.

Strengths.

- **Self-Correcting Mechanism**: The three-action framework provides robustness against retrieval failures, with the Ambiguous action serving as a crucial moderating mechanism.
- **Adaptive Precision/Recall**: Question-type-aware thresholds optimize for completeness (enumeration) or precision (factual) as needed.
- **Semantic Chunking**: Structure-aware segmentation preserves coherent concepts, addressing the "scattered takeaways" problem in fixed-size chunking.
- **Graceful Degradation**: Web search fallback ensures useful answers even when internal retrieval fails completely.
- **Plug-and-Play Nature**: CRAG can be integrated with various RAG systems without architectural changes, as demonstrated by Self-CRAG improvements.
- **Transparency**: Action classification and citation tracking provide interpretable signals for debugging and user trust.

Weaknesses.

- **API Dependency**: GPT-4o-mini evaluation and GPT-4o generation incur API costs (~$0.02 per 7 questions) and require network connectivity.
- **Threshold Sensitivity**: Despite adaptive profiles, edge cases between question types may receive suboptimal settings. The original paper notes high sensitivity to evaluator accuracy without the Ambiguous action.
- **Web Search Noise**: DuckDuckGo results occasionally include less authoritative sources; Wikipedia-preferential filtering (as in the original paper) could improve reliability.
- **Latency**: Average 11.8s per question may be too slow for real-time applications due to multi-stage evaluation.

- **Evaluator Training**: The original T5-based evaluator requires fine-tuning on domain-specific question-document pairs for optimal performance.

Use Cases.
Our Question-Adaptive CRAG is particularly well-suited for:

1. **Executive Commentary Questions**: Queries about annual letters from BlackRock leadership achieved high quality scores (1.00) with 14–15 citations, demonstrating effective handling of structured long-form documents.
2. **Membership/List Questions**: "Who are the GEC members?" benefited from enumeration-mode's lenient thresholds and web search augmentation.
3. **Market Outlook Queries**: Questions about 2025 views triggered AMBIGUOUS actions, appropriately combining internal corpus with current web information.
4. **Comparative Analysis**: Comparison questions automatically received balanced retrieval settings, though performance depends heavily on both entities being well-represented in the corpus.
5. **Knowledge-Intensive Tasks**: Any application where retrieval quality directly impacts generation quality, particularly when the corpus may be incomplete or outdated.

4.5 LightRAG

LightRAG (Light Retrieval-Augmented Generation), proposed by Guo et al. (2024), is a graph-based RAG framework that constructs a knowledge graph from the corpus and performs retrieval through vector similarity matching on entity and relation embeddings. Unlike GraphRAG, as proposed by Edge et al. (2024), which relies on computationally expensive community detection algorithms such as Leiden clustering to generate hierarchical summaries, LightRAG adopts a lightweight, embedding-centric approach that enables faster indexing, more efficient retrieval, and seamless incremental updates. Our implementation adapts the LightRAG paradigm for the BlackRock corporate knowledge base, with domain-specific prompt engineering for entity and relation extraction, semantic keyword annotation for relations, and a dual-mode retrieval system combining local similarity search with global graph traversal.

Architecture Overview.
The implementation comprises seven modular components organized into three stages:

1. **Data Processing Stage**: `BlackRockLoader` for JSON article ingestion, `TextPreprocessor` for text normalization (removing URLs, emails, special characters, and normalizing whitespace), and `TextChunker` for semantic document segmentation.
2. **Indexing Stage**: `EntityRelationExtractor` using GPT-4o with domain-specific prompts for knowledge extraction, `KnowledgeGraphBuilder` for graph construction with Jaccard-based entity deduplication, and `TextEmbedder` for generating dense vector representations.
3. **Retrieval Stage**: `LocalRetriever` for vectorized similarity search, `GlobalRetriever` for BFS-based graph traversal, `HybridRetriever` for combining both approaches, and `QueryProcessor` for context formatting and response generation.

LightRAG's key design principle is **embedding-centric retrieval**: all graph elements (entities, relations, chunks) are pre-embedded, enabling $O(n)$ similarity search without runtime community detection or hierarchical summarization, while supporting $O(1)$ incremental up-

dates for new documents.

Text Preprocessing and Semantic Chunking.
Raw articles undergo a two-stage preprocessing pipeline before knowledge extraction:

Text Normalization. The `TextPreprocessor` applies regex-based cleaning to remove noise while preserving semantic content: (1) non-breaking spaces (`\xa0`) are replaced with regular spaces; (2) URLs and email addresses are stripped; (3) special characters except punctuation are removed; (4) multiple whitespace sequences are collapsed to single spaces. Title, subtitle, and content fields are processed independently.

Semantic Chunking. The `TextChunker` implements an adaptive chunking strategy based on inter-sentence semantic similarity. Documents are first split into sentences using regex patterns that respect abbreviations (e.g., "Dr.", "U.S."). All sentence embeddings are computed in a single batch API call for efficiency. For each adjacent sentence pair (s_i, s_{i+1}), cosine similarity is computed:

$$\text{sim}(s_i, s_{i+1}) = \frac{\mathbf{e}_i \cdot \mathbf{e}_{i+1}}{\|\mathbf{e}_i\| \|\mathbf{e}_{i+1}\|} \tag{1}$$

where $\mathbf{e}_i$ denotes the embedding of sentence s_i. A chunk boundary is inserted when similarity drops below threshold $\tau = 0.5$, subject to size constraints: minimum 200 characters (to avoid trivial chunks) and maximum 2000 characters (to ensure processability). This approach preserves topical coherence within chunks while creating natural semantic boundaries.

For documents where the LLM adapter is unavailable, a fallback fixed-size chunking strategy splits text at paragraph or sentence boundaries while maintaining configurable overlap (default: half of minimum chunk size) between consecutive chunks to preserve context continuity.

Domain-Specific Prompt Engineering.
A key innovation in our implementation is the use of carefully designed prompts tailored for BlackRock financial content. The extraction pipeline employs two specialized prompts:

Entity Extraction Prompt. The prompt defines 15 domain-specific entity types organized into seven categories:

- *Organizations & Divisions*: `Organization` (external companies, institutions), `BlackRockDivision` (internal business units like iShares, Aladdin, BlackRock Foundation)
- *People & Roles*: `Person` (individuals with titles), `ExecutiveRole` (leadership positions)
- *Financial Products & Services*: `FinancialProduct` (specific products like ETFs), `AssetClass` (investment categories), `Technology` (platforms like Aladdin)
- *Programs & Initiatives*: `Program` (specific initiatives), `ESGInitiative` (sustainability programs)
- *Locations*: `Location` (geographic entities)
- *Financial Metrics*: `MonetaryAmount` (dollar figures with context), `Percentage`, `FinancialMetric` (KPIs like AUM, EPS)
- *Temporal & Statistical*: `Date` (specific dates/periods), `Statistic` (numerical data points), `IndustryTerm` (domain concepts)

The prompt includes explicit extraction guidelines emphasizing specificity ("extract full product names like 'iShares Core S&P 500 ETF' not just 'ETF' "), contextual completeness ("for amounts, include what they represent"), and description requirements for each entity type. A worked example demonstrates expected output format.

Relation Extraction Prompt. Given extracted entities and source text, the relation prompt defines 30+ relationship types across eight categories:

- *Corporate & Business*: `ACQUIRED`, `PARTNERED_WITH`, `MERGED_WITH`, `DIVESTED_FROM`
- *Product & Service*: `LAUNCHED`, `OFFERS`, `MANAGES`, `POWERED_BY`, `TRACKS`
- *Investment & Financial*: `INVESTED_IN`, `ALLOCATED_TO`, `HOLDS`, `DISTRIBUTED`
- *Organizational*: `PART_OF`, `DIVISION_OF`, `LEADS`, `EMPLOYED_BY`, `REPORTS_TO`
- *Program & Initiative*: `FUNDED_BY`, `SUPPORTS`, `BENEFITS`, `IMPLEMENTS`
- *Geographic & Temporal*: `LOCATED_IN`, `OPERATES_IN`, `HEADQUARTERED_IN`, `DATED`, `REPORTED_IN`
- *Financial Metric*: `VALUED_AT`, `GREW_BY`, `PRICED_AT`, `YIELDED`
- *ESG & Governance*: `VOTED_ON`, `COMMITTED_TO`, `ALIGNED_WITH`

Critically, each relation is extracted with both a natural language **description** (1–2 sentences explaining the relationship context, amounts, and timeframe) and 3–5 **semantic keywords** capturing the relationship's core concepts. For example:

```
{
  "source": "BlackRock Foundation",
  "target": "Technical College System of Georgia",
  "relation_type": "PARTNERED_WITH",
  "description": "BlackRock Foundation partnered with Technical
    College System of Georgia to enhance workforce development
    programs in Georgia, focusing on economic mobility.",
  "keywords": ["partnership", "workforce development",
    "economic mobility", "Georgia", "education"]
}
```

These keywords provide additional semantic signals that are preserved in the knowledge graph and could enable future keyword-based retrieval or filtering mechanisms.

Knowledge Graph Construction.

The `KnowledgeGraphBuilder` constructs a directed graph from extracted entities and relations using NetworkX:

Entity Deduplication. Entities extracted from different chunks may refer to the same real-world concept with slight variations. The builder applies Jaccard similarity on tokenized entity names:

$$J(e_1, e_2) = \frac{|W(e_1) \cap W(e_2)|}{|W(e_1) \cup W(e_2)|} \tag{2}$$

where $W(e)$ is the set of lowercase tokens in entity name e. Entities with identical types and $J \geq 0.8$ are merged: their descriptions are concatenated, and source references are consolidated into a unified `source_chunks` list.

Graph Assembly. Each unique entity becomes a node with attributes: `entity_name`, `entity_type`, `description`, and `source_chunks`. Each relation becomes a directed edge with attributes: `source`, `target`, `relation_type`, `description`, `keywords`, and `source_chunks`. The graph is serialized to JSON for persistence and downstream embedding.

Embedding Generation.

All graph elements are embedded using OpenAI's `text-embedding-3-large` model, producing 3072-dimensional dense vectors. The `TextEmbedder` implements batch processing (up to 2048 texts per API call) with MD5-based caching to minimize redundant computation:

Entity Embedding. Each entity is represented as a concatenated string:

```
"{entity_name} ({entity_type}): {description}"
```

For example: "BlackRock Foundation (BlackRockDivision): BlackRock's philanthropic arm providing grants for education and economic opportunity". This format captures identity, category, and semantic context in a single embedding.

Relation Embedding. Each relation is represented as:

```
"Relationship from {source} to {target} ({relation_type}): {description}"
```

For example: "Relationship from BlackRock Foundation to Technical College System of Georgia (PARTNERED_WITH): BlackRock Foundation partnered with Technical College System of Georgia to enhance workforce development programs in Georgia, focusing on economic mobility and in-demand job skills." Note that the semantic **keywords** extracted during relation extraction are stored in the graph but not included in the embedding text—they are preserved for potential future use in hybrid retrieval or filtering.

Chunk Embedding. Text chunks are embedded directly from their raw content without additional formatting, enabling semantic similarity search against the original document text.

The embedding cache uses MD5 hashes of normalized input text (whitespace-collapsed, newlines replaced) as keys, enabling efficient re-indexing when documents are updated or prompts are modified.

Dual-Mode Retrieval System.

At query time, LightRAG supports three retrieval modes implemented across two specialized retrievers:

Local Retrieval (`LocalRetriever`) performs vectorized similarity search across all graph elements. Given a query embedding $\mathbf{q} \in \mathbb{R}^{3072}$, the retriever:

1. Loads all entity embeddings into a matrix $\mathbf{E} \in \mathbb{R}^{n \times 3072}$
2. Computes batch cosine similarities: $\mathbf{s} = \frac{\mathbf{E}\mathbf{q}}{\|\mathbf{E}\|_{\text{row}} \cdot \|\mathbf{q}\|}$
3. Returns top-k entities by similarity (default $k = 5$)
4. Repeats for relations and chunks independently

This vectorized implementation using NumPy enables efficient $O(n)$ retrieval without explicit loops over individual embeddings. Each retrieved element includes its similarity score for downstream ranking or filtering.

Global Retrieval (`GlobalRetriever`) performs graph traversal to capture structural context:

1. *Seed Selection*: Identify top-k seed entities (default $k = 3$) by vector similarity to query embedding
2. *BFS Expansion*: Starting from seeds, perform breadth-first traversal collecting both successors and predecessors at each hop
3. *Subgraph Extraction*: After d hops (default $d = 2$), extract all reached entities and edges connecting them

The bidirectional traversal (both successors and predecessors) ensures that contextually relevant entities are captured regardless of edge direction in the original extraction.

Hybrid Retrieval (`HybridRetriever`) combines both approaches:

1. Execute local retrieval to obtain top-k entities, relations, and chunks by similarity
2. Execute global retrieval to obtain seed-expanded subgraph
3. Merge results with deduplication: entities by ID, relations by (source, target, type) tuple

4. Return unified context containing entities, relations, and chunks

Default parameters: local retrieval uses $k = 5$ for entities, relations, and chunks; global retrieval uses 3 seed entities with 2-hop expansion.

Query Processing and Response Generation.
The `QueryProcessor` formats retrieved context into a structured prompt for GPT-4o:

```
Context:
Entities:
1. {name} ({type}): {description}
2. ...

Relations:
1. {source} {relation_type} {target}: {description}
2. ...

Text Chunks:
1. [Source: {source_id}] {content}
2. ...

Question: {user_query}
```

A domain-specific system prompt establishes the assistant's expertise: "You are an expert assistant in the financial domain, specializing in providing detailed and professional answers about BlackRock company, its products, services, and reports." Query results are cached using MD5 hashes of (query, context) pairs to avoid redundant LLM calls for repeated questions.

Comparison with GraphRAG.
LightRAG offers several architectural advantages over Microsoft's GraphRAG framework:

Aspect	LightRAG	GraphRAG
Community Detection	Not required	Leiden clustering
Summarization	None (direct embedding)	Hierarchical community summaries
Relation Representation	Embedding + keywords	Embedding only
Indexing Complexity	$O(n)$	$O(n \log n)$ to $O(n^2)$
Incremental Update	$O(1)$ append	Requires re-clustering
Query Latency	Single vector search + BFS	Multi-level community traversal
Storage Overhead	Embeddings + keywords	Embeddings + summaries

Table 8: Architectural Comparison: LightRAG vs. GraphRAG

Strengths.

- **Domain-Specific Extraction**: Carefully engineered prompts with 15 entity types and 30+ relation types tailored for financial content ensure high-quality knowledge extraction from BlackRock documents.
- **Rich Relation Representation**: Each relation stores both a natural language description and semantic keywords, providing multiple signals for understanding relationship semantics.

- **Computational Efficiency**: Vectorized similarity computation and absence of community detection reduce indexing time from hours to minutes. Batch embedding API calls (up to 2048 texts) minimize latency.
- **Incremental Updates**: New documents can be processed independently—entities and relations are extracted, deduplicated against existing nodes (via Jaccard similarity), embedded, and appended without rebuilding the graph.
- **Flexible Retrieval**: The three-mode architecture (local/global/hybrid) allows query-adaptive retrieval strategies based on question type.
- **Full Provenance**: Every entity and relation maintains `source_chunks` references, enabling users to trace answers back to original documents.

Weaknesses.

- **No Hierarchical Abstraction**: Unlike RAPTOR or GraphRAG, LightRAG does not generate document-level or corpus-level summaries. Thematic queries requiring holistic understanding may retrieve fragmented evidence.
- **Extraction Quality Dependency**: System performance is bounded by GPT-4o's ability to correctly identify entities and relations. Ambiguous or inconsistent extractions propagate to retrieval quality.
- **Fixed Traversal Depth**: Global retrieval uses fixed 2-hop expansion, which may be insufficient for queries requiring longer reasoning chains or excessive for simple entity lookups.
- **Embedding-Only Matching**: All retrieval relies on embedding similarity in a single 3072-dimensional space, which may conflate semantically distinct concepts with similar surface representations.

Use Cases.
LightRAG is particularly well-suited for applications where:

- **Frequent Corpus Updates**: Knowledge bases requiring continuous document ingestion benefit from $O(1)$ incremental updates without re-indexing.
- **Entity-Centric Queries**: Questions about specific entities, their attributes, and relationships leverage the structured graph representation with typed entities and relations.
- **Multi-Hop Reasoning**: Queries requiring traversal across entity relationships benefit from global retrieval with configurable BFS expansion depth.
- **Domain-Specific Corpora**: The prompt engineering approach allows customization for different domains by modifying entity types, relation types, and extraction guidelines.
- **Provenance Requirements**: Applications requiring explicit source attribution can leverage `source_chunks` metadata for full traceability.

4.6 LinearRAG

Recent advances in Graph-based Retrieval-Augmented Generation (GraphRAG) aim to improve multi-hop reasoning by explicitly modeling relational structures over large text corpora. However, extracting accurate and stable relational triples from natural language remains computationally expensive and linguistically fragile, often introducing noise, factual inconsistencies,

and structural instability into the constructed graphs. These limitations substantially constrain the scalability and robustness of relation-heavy GraphRAG systems, particularly in large enterprise domains.

To address this bottleneck, Zhuang et al. (2025) proposes **Linear Graph-based Retrieval-Augmented Generation (LinearRAG)**, a relation-free graph-based retrieval framework that preserves multi-hop reasoning capability while completely avoiding explicit relation extraction and LLM-token-intensive indexing operations. LinearRAG demonstrates that effective graph-based retrieval can be achieved through lightweight structural containment alone, rather than relying on noisy symbolic relations.

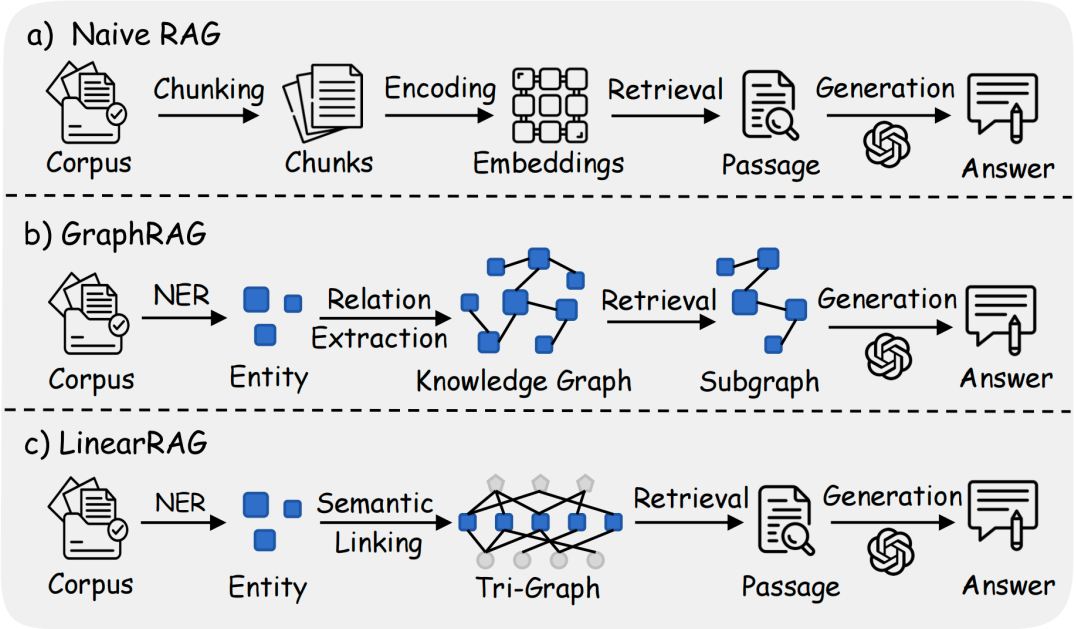

Figure 2: High-level comparison illustrating the LinearRAG paradigm (Figure 1(c) in the original paper), contrasting naive RAG, relation-based GraphRAG, and the proposed relation-free design.

Architecture Overview.
At a high level, LinearRAG consists of two decoupled components: (i) an *offline indexing phase* that constructs a lightweight semantic graph without relation extraction, and (ii) an *online retrieval phase* that performs structure-aware, multi-hop ranking through constrained graph traversal and importance aggregation.

Tri-Graph Construction. During indexing, LinearRAG constructs a lightweight **Tri-Graph** that captures document structure without modeling explicit semantic relations. Documents are segmented into passages and sentences, while named entities are extracted using standard NER tools (e.g., spaCy). Based on simple containment, three node types are created: *entities*, *sentences*, and *passages*. Edges are added only when an entity appears in a sentence, or when a sentence belongs to a passage. In addition, semantically similar passages may be connected via embedding similarity.

This construction yields sparse adjacency matrices whose size scales linearly with corpus size. Because no summarization, abstraction, or relation inference is performed, Tri-Graph construction is token-free, lossless, and robust to linguistic variation, making it suitable for large and frequently updated enterprise corpora.

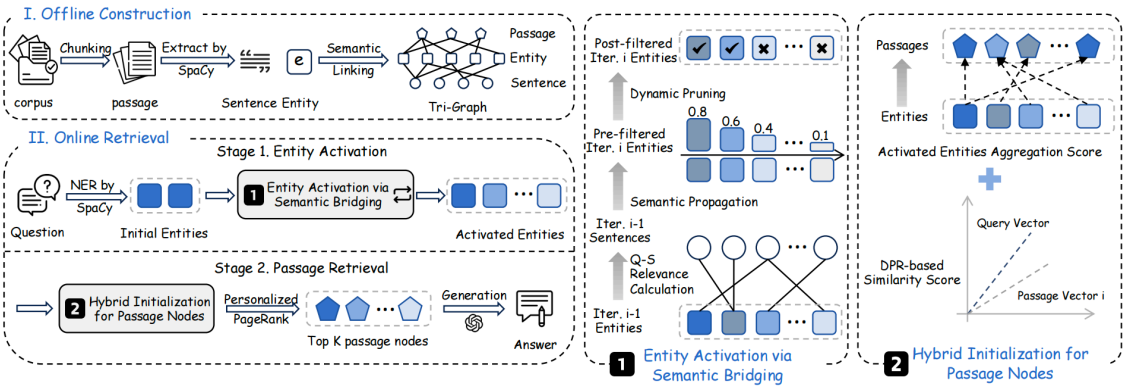

Figure 3: Overall LinearRAG pipeline (Figure 3 in the original paper), illustrating offline Tri-Graph construction and the two-stage online retrieval process consisting of local semantic bridging and global importance aggregation.

Two-Stage Retrieval. At query time, LinearRAG employs a two-stage retrieval strategy designed to support multi-hop reasoning while avoiding uncontrolled graph expansion.

(1) Local Semantic Bridging. Retrieval begins by identifying entities explicitly mentioned in the query. LinearRAG then performs iterative propagation over the entity–sentence subgraph, allowing repeated transitions between entities and sentences. This process activates not only the queried entities but also implicitly related intermediate entities that are structurally and semantically connected.

Importantly, propagation is guided by query–sentence embedding similarity and controlled through dynamic pruning. Rather than expanding indiscriminately, low-confidence or semantically drifting nodes are pruned to ensure that the activated subgraph forms a coherent semantic neighborhood centered on the query intent.

(2) Global Importance Aggregation. In the second stage, the activated entities serve as seed nodes for Personalized PageRank on the passage–entity graph. Each passage receives an initial score based on both its dense similarity to the query and its connectivity to the activated entities. Personalized PageRank then aggregates structural importance across the graph, producing a ranking that reflects multi-hop relevance rather than isolated semantic similarity.

This global aggregation step enables LinearRAG to retrieve passages that collectively support the query, avoiding the fragmentation commonly observed in purely dense retrieval systems.

Discussion. By combining dense retrieval with constrained graph traversal, LinearRAG achieves robust multi-hop retrieval without explicit relation modeling. Its relation-free design eliminates a major source of noise in GraphRAG systems while retaining interpretability and scalability. Although LinearRAG may be sensitive to NER quality and introduces greater implementation complexity than vanilla RAG, it remains substantially simpler and more stable than relation-heavy or hierarchical alternatives such as LightRAG. Overall, LinearRAG provides a principled and practical solution for enterprise-scale, structure-aware retrieval under strict cost and reliability constraints.

5 Results, Deliverables, and Insights

5.1 Evaluation Criteria

To compare different Retrieval-Augmented Generation (RAG) variants on BlackRock-domain questions, we evaluate each system using three error-rate metrics. Each metric is reported as the fraction of failures over a fixed set of 7 evaluation questions.

Wrong Category Rate (WC). The proportion of retrieved chunks that are off-topic in terms of BlackRock website domain/category (i.e., retrieved context comes from an incorrect BlackRock domain for the query).

Context Deficiency Rate (CD). The proportion of model outputs that indicate insufficient context or provide no useful or actionable information due to missing evidence in the retrieved context.

Incorrect Link Retrieval Rate (IL). The proportion of retrievals where the returned sources originate from wrong, non-optimal, or non-target links, leading to degraded grounding quality.

5.2 Results

Table 9 summarizes the evaluation results across multiple RAG systems. Among the fully implemented and comparable baselines, graph-based approaches demonstrate clear advantages in retrieval quality. LinearRAG achieves perfect scores across all three error metrics, but at a significant computational cost: it is slow during knowledge graph construction and remains relatively slow at retrieval time due to multi-hop graph traversal and global structure aggregation.

LightRAG, while belonging to the graph-based RAG family, exhibits a different computational profile. Compared to non-graph baselines such as Vanilla RAG, CRAG, and RAPTOR, LightRAG is relatively computationally expensive overall. However, it is substantially more efficient than traditional GraphRAG-style approaches. In particular, the primary computational bottleneck of LightRAG lies in the graph construction stage, whereas retrieval at query time is fast due to its lightweight graph structure and localized retrieval strategy.

Retrieval-targeted methods such as RAPTOR and CRAG improve over Vanilla RAG by reducing off-topic retrievals, but they do not fully eliminate context deficiency or incorrect link retrievals. LightRAG is currently evaluated on a smaller corpus (20 articles), and therefore its results are not directly comparable to the fully implemented systems.

Table 9: Evaluation results on 7 BlackRock-domain questions. WC = Wrong Category Rate, CD = Context Deficiency Rate, IL = Incorrect Link Retrieval Rate.

Model	Status	WC	CD	IL	Notes
Vanilla RAG	Fully Implemented	1/7	1/7	2/7	Base model
CRAG	Fully Implemented	1/7	0/7	2/7	Retrieval-targeted model
RAPTOR	Fully Implemented	0/7	1/7	0/7	Retrieval-targeted; tree-based
LinearRAG	Fully Implemented	0/7	0/7	0/7	Graph RAG family
LightRAG	20 Articles	0/7	0/7	0/7	Not comparable; Graph RAG family

Summary. Based on the above metrics, the overall performance ranking among the fully comparable systems is:

$$\text{LinearRAG} > \text{RAPTOR} > \text{CRAG} > \text{Vanilla RAG}.$$

5.3 Top-K Retrieval Analysis and Evaluation Directions

As part of our retrieval analysis, we compiled the top-K ranked text chunks for each model with $K \in \{5, 10, 20\}$. This compilation was conducted consistently across all evaluated RAG variants and forms the basis for subsequent qualitative inspection and quantitative evaluation of retrieval quality.

This Top-K analysis serves three practical objectives.

First, it addresses the asymmetry in ground truth availability between research and business settings in real-world RAG systems. In our experimental environment, ground truth and complete list of links corresponding to a specific question are not available, which prevents rigorous quantitative evaluation under the evaluation criteria described earlier. However, BlackRock maintains internal ground truth and a complete list of authoritative document links. By providing BlackRock with the Top-K retrieved text chunks produced by each model under multiple values of K, the retrieved results can be directly compared against these internal references, enabling systematic assessment of retrieval effectiveness and stability in a production-relevant context.

Second, this approach accounts for the one-to-many relationship between queries and relevant information. Many real-world questions cannot be answered by a single document and instead require synthesizing evidence distributed across multiple sources. Evaluating Top-5, Top-10, and Top-20 retrieval results allows assessment of each models ability to capture the breadth of relevant information, rather than focusing solely on whether the top-ranked result is correct.

Third, the compiled Top-K results provide a robust foundation for subsequent, more comprehensive evaluation. When combined with BlackRocks internal ground truth, these results support comparative analysis of retrieval strategies across different question types and facilitate identification of both strengths and remaining limitations in retrieval quality.

To formalize this evaluation, we recommend adopting two standard information retrieval metrics: Mean Reciprocal Rank (MRR) and Mean Average Precision (MAP).

Mean Reciprocal Rank (MRR). MRR measures how early the first relevant result appears in the ranked list. For a set of queries $\mathcal{Q}$, MRR is defined as:

$$\mathrm{MRR} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \frac{1}{\mathrm{rank}_q},$$

where rank_q denotes the rank position of the first relevant retrieved chunk for query q. The value of MRR lies in the range $[0, 1]$, with higher values indicating better early precision. MRR is particularly well suited for question-answering and RAG scenarios where a single high-quality passage may be sufficient to answer a query.

Mean Average Precision (MAP). MAP provides a more comprehensive assessment by considering all relevant documents for each query. For a given query q, the Average Precision (AP) is defined as:

$$\mathrm{AP}(q) = \frac{1}{|\mathcal{R}_q|} \sum_{k=1}^{N} \mathrm{P}_q(k) \cdot \mathbf{1}_q(k),$$

where $\mathcal{R}_q$ is the set of relevant documents for query q, $\mathrm{P}_q(k)$ denotes the precision at rank k, $\mathbf{1}_q(k)$ is an indicator function equal to 1 if the item at rank k is relevant and 0 otherwise, and N is the total number of retrieved items. MAP is then computed as:

$$\mathrm{MAP} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \mathrm{AP}(q).$$

MAP evaluates overall ranking quality and rewards systems that consistently rank all relevant passages higher, making it especially important for complex queries requiring evidence from multiple sources.

Together, MRR and MAP provide complementary perspectives on retrieval performance. MRR evaluates how quickly a system can surface a useful passage, while MAP measures the completeness and overall quality of the ranking. As a next step, we recommend applying these metrics to the compiled top-K results to establish quantitative benchmarks and guide future retrieval improvements.

6 Maintainability and Future Directions

6.1 Index Management and Maintainability

This section presents the architectural design and implementation details of five retrieval-augmented generation (RAG) frameworks evaluated in this paper. Each framework implements a standardized index interface while accommodating architecture-specific requirements for document ingestion, retrieval, and maintenance operations.

Vanilla RAG Vanilla RAG establishes the baseline retrieval-augmented generation architecture. The indexing pipeline segments source documents into fixed-size chunks with configurable overlap, transforms each chunk into a dense vector representation using a pre-trained embedding model, and persists the resulting vectors in a high-dimensional index with associated document metadata. Vanilla RAG systems exhibit relatively good maintainability characteristics due to their modular architecture. The separation of retrieval and generation components allows for independent updates to either subsystem. Adding new documents to the knowledge base requires only embedding the new content and updating the vector database, without necessitating model retraining. However, maintaining optimal performance requires periodic re-evaluation of chunk sizes, embedding models, and retrieval parameters as the document corpus evolves. The embedding model choice is critical, as changing the embedding model requires re-embedding the entire corpus. Additionally, organizations must establish processes for data quality assurance to prevent the introduction of counterfactual or false information into the vector database, as noted in challenges amplified by RAG (Klesel and Wittmann, 2025).

Index Operations. The core interface exposes three primary operations: (1) `upsert(doc_id, chunks)` embeds input chunks and stores the resulting vectors with their associated document identifier; (2) `delete(doc_id)` removes all vector entries corresponding to the specified document; and (3) `query(embedding, top_k)` executes approximate nearest neighbor search to retrieve the k most semantically similar chunks.

Update Mechanism. Document modifications necessitate only a localized upsert operation targeting the affected document identifier. The absence of inter-document dependencies eliminates cascading updates, yielding minimal maintenance overhead.

LinearRAG LinearRAG augments the Vanilla RAG framework by encoding explicit structural metadata within the index. Each chunk maintains positional coordinates comprising document identifier, section index, and intra-section offset, enabling structure-aware retrieval and context expansion.

Index Operations. The interface extends Vanilla RAG with positional metadata: (1) `upsert(doc_id, chunks, positions)` stores chunks with associated $(doc_id, section_idx, chunk_idx)$ coordinate tuples; (2) `delete(doc_id)` purges all vectors and corresponding positional metadata; and (3) `query(embedding, top_k, context_window)` retrieves semantically similar chunks with optional expansion to include structurally adjacent segments.

Update Mechanism. Index updates mirror the Vanilla RAG procedure with supplementary maintenance of positional indices. Structural recomputation remains confined to the modified document, preserving system-wide efficiency.

CRAG (Corrective RAG) CRAG systems exhibit moderate maintainability complexity. The modular architecture (evaluator, refiner, web searcher, generator) allows for independent component updates. Adding new documents requires only embedding and indexing without model retraining. However, maintaining optimal performance requires periodic calibration of action thresholds as the corpus evolves. The question-type detection patterns may need updating as new query patterns emerge. Organizations must also monitor web search quality and potentially implement source filtering to maintain answer reliability.

Index Operations. Index operations maintain parity with Vanilla RAG: (1) `upsert(doc_id, chunks)` and (2) `delete(doc_id)` function identically. The `query` operation extends the base retrieval with relevance scoring, triggering query reformulation or external knowledge augmentation when retrieval confidence falls below configurable thresholds.

Update Mechanism. Document updates conform to the Vanilla RAG protocol. The corrective evaluation module maintains no persistent state dependent on index contents, thereby requiring no additional maintenance upon document modification.

RAPTOR RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval) constructs a hierarchical index through iterative clustering and abstractive summarization. The resulting tree structure positions original document chunks at leaf nodes, with internal nodes containing machine-generated summaries that capture increasingly abstract representations of their descendant content.

RAPTOR presents more complex maintainability challenges compared to vanilla RAG due to its hierarchical architecture. Adding new documents requires not only embedding the new content but also re-running the clustering and summarization processes, which involve: (1) determining how new leaf nodes integrate into existing clusters using GMM, (2) potentially triggering re-summarization of affected parent nodes, and (3) updating embeddings throughout affected branches of the tree. The system's dependency on multiple componentsSBERT embedding, GMM clustering with UMAP dimensionality reduction, BIC-based cluster selection, and GPT-3.5-turbo summarizationcreates multiple potential points of failure and version control requirements.

Organizations implementing RAPTOR must establish sophisticated MLOps capabilities to manage the tree construction pipeline, monitor summarization quality (including the 4% hallucination rate), and handle the recursive nature of the system (Klesel and Wittmann, 2025). Parameter tuning involves optimization of clustering parameters (UMAP neighbors: 10, BIC thresholds), summarization prompts, and retrieval strategies (collapsed tree token limit: 2000). Unlike vanilla RAG where the embedding model can be changed by re-embedding all documents independently, RAPTOR requires coordinated re-embedding of both leaf nodes and all summary nodes.

However, RAPTOR benefits from modular design where individual components can be upgraded independentlyfor instance, replacing GPT-3.5-turbo with a different summarization model without changing the clustering mechanism. The tree structure can be serialized and cached, amortizing build costs (30–45 minutes, approximately $2.00) across multiple query sessions. For organizations with static or slowly evolving document collections, the upfront construction cost may be acceptable given the performance improvements on complex reasoning tasks.

Index Operations. The hierarchical construction process comprises: (1) `build_tree(doc_id, chunks)` recursively applies clustering algorithms to group semantically related chunks, generating parent summaries via large language model inference at each aggregation level; (2)

`delete(doc_id)` removes leaf-level chunks and propagates deletion through all dependent summary nodes; and (3) `query(embedding, top_k, traversal)` supports both collapsed retrieval across all hierarchy levels and structured tree traversal strategies.

Update Mechanism. Document modifications trigger re-clustering of affected leaf nodes and necessitate regeneration of summary content for all ancestor nodes along the affected paths. This hierarchical dependency structure renders RAPTOR the most maintenance-intensive architecture, as localized modifications may propagate through multiple abstraction levels.

LightRAG LightRAG implements a knowledge graph-augmented retrieval architecture that extracts structured entity-relationship representations from unstructured text. The dual-index design combines vector similarity search with graph traversal capabilities to enable semantically informed, structure-aware retrieval.

LightRAG exhibits favorable maintainability characteristics due to its modular, stateless design. Each pipeline stage operates independently with JSON-based intermediate outputs (`chunks.json`, `extraction_results.json`, `knowledge_graph.json`, `knowledge_graph_with_embeddings.js` The multi-level caching infrastructure (entity extraction cache, embedding cache, query cache) enables efficient re-processing when prompts are updated or models are changed.

Adding new documents requires: (1) preprocessing and chunking, (2) entity/relation extraction with cache lookup, (3) graph merging with Jaccard deduplication, (4) embedding new elements with cache lookup, and (5) appending to existing JSON files. Changing the embedding model requires re-embedding the entire corpus, as similarity scores are only meaningful within a consistent embedding space.

For production deployments, the JSON-based storage should be migrated to a vector database (e.g., Pinecone, Weaviate, Milvus) to support approximate nearest neighbor search at scale. The current exact similarity computation becomes prohibitive for graphs exceeding 10^5 entities.

Index Operations. The graph construction pipeline comprises: (1) `extract_graph(doc_id, text)` applies named entity recognition and relation extraction via large language model inference; (2) `upsert_entities(doc_id, entities)` incorporates extracted entities as graph nodes with associated vector embeddings; (3) `upsert_relations(doc_id, relations)` establishes typed edges encoding inter-entity relationships; (4) `delete(doc_id)` removes all entities and relationships attributed to the specified document; and (5) `query(embedding, top_k, hops)` executes hybrid retrieval combining vector similarity with configurable graph traversal depth.

Update Mechanism. LightRAG supports incremental graph modification. Document updates affect only the entities and relationships directly attributable to the modified content. The graph topology accommodates additive changes without necessitating global recomputation, maintaining favorable scalability characteristics.

Maintainability Analysis Table 10 provides a comparative assessment of update complexity and operational maintainability across the evaluated RAG architectures.

6.2 Future Directions

6.2.1 Graph-Based RAG Recommendations

While the proposed system already integrates a variety of retrieval-enhanced paradigms,there remains substantial opportunity to further enhance retrieval robustness,semantic precision, and scalability by incorporating complementary advances fromrecent Graph-based RAG literature. In particular, emerging methods such asE2GraphRAG, HippoRAG, and GFM-RAG introduce orthogonal design choices. For example:

- **E^{2}GraphRAG** emphasizes efficient multi-hop reasoning through tighter entity-sentence linking;

Table 10: Comparative Analysis of RAG Architecture Maintainability

Architecture	**Update Operations Required**	**Maintainability**
Vanilla RAG	Vector upsert/delete for affected document ID only	Very Easy
LinearRAG	Vanilla RAG operations with local $(doc, section)$ ordering maintenance	Very Easy
CRAG	Vanilla/Linear operations; corrective evaluation logic requires no update	Moderate
RAPTOR	Re-clustering of affected chunks; regeneration of local and ancestor summaries	Heavy
LightRAG	Incremental entity and relationship graph updates	Easy

- **HippoRAG** enhances global context comprehension using hybrid summarization + graph navigation;
- **GFM-RAG** leverages high-quality foundation-model embeddings to build more semantically precise graph structures.

Model	**Core Idea**	**Technical Mechanism**	**Strengths / Ideal Use Cases**
E^2GraphRAG	Entity-enhanced graph retrieval focusing on efficient multi-hop reasoning.	Builds tighter entity–sentence–passage connections using robust NER; avoids complex relation extraction; performs entity-driven graph activation.	Best for entity-rich corpora (e.g., financial instruments, macro factors); strong for cross-document conceptual linking and precise multi-hop chains.
HippoRAG	Hybrid hierarchical + graph-based retrieval capturing both global context and local detail.	Combines summary-tree hierarchy (high-level themes) with graph traversal between entities, sections, and passages.	Ideal for large, structured corpora (e.g., insights macro fixed income); excellent for thematic reasoning and long-range context integration.
GFM-RAG	Graph Foundation Modelbased RAG with learned semantic relations.	Uses graph-transformer foundation models to infer entity/entity, sentence/sentence, and concept/concept relations; builds high-quality semantic graphs.	Strongest semantic precision; ideal for abstract domains (e.g., ESG, macro research) and capturing deeper cross-topic conceptual relationships.

Table 11: Comparison of three advanced graph-based RAG methods and their characteristics.

Incorporating selected ideas from these modelsparticularly improved entity disambiguation,

graph sparsification, or adaptive graph traversalcould strengthen the systems ability to handle complex thematic questions and cross-document reasoning.

6.2.2 Adaptive and Hybrid RAG Selection

Beyond incorporating additional graph-based retrieval techniques, a promising future direction is to move from a single fixed RAG pipeline toward an adaptive and hybrid RAG system that dynamically selects or combines retrieval strategies based on query characteristics.

Different RAG variants exhibit distinct strengths depending on the structure and intent of the query. For example, simple factual or entity-centric questions may be well served by dense retrieval or LinearRAG-style entity propagation, while thematic, cross-document, or abstract questions may benefit from hierarchical or foundation-model-based graph reasoning. Rather than enforcing a one-size-fits-all retrieval strategy, the system could first analyze the query to determine its semantic complexity, entity density, and required reasoning depth, and then route the query to the most appropriate RAG pipeline.

One possible design is a query-aware routing layer that performs lightweight query analysissuch as entity count, embedding dispersion, or predicted reasoning depth to select among multiple retrieval backends (e.g., vanilla RAG, LinearRAG, or other graph-based variants). This routing decision could be rule-based initially and later extended to a learned policy trained on historical queryresponse performance.

An alternative approach is hybrid retrieval composition, where multiple RAG pipelines are executed in parallel and their retrieved contexts are merged or re-ranked. For instance, dense retrieval could provide high-recall candidates, while graph-based retrieval contributes structurally important passages. A downstream re-ranking or aggregation step could then reconcile these signals to produce a final context set that balances semantic relevance and global coherence.

Such adaptive or hybrid strategies would allow the system to exploit the complementary strengths of different RAG paradigms while mitigating their individual weaknesses. This direction is particularly well aligned with enterprise knowledge systems, where query types vary widely and retrieval robustness, interpretability, and cost efficiency must be balanced dynamically.

References

Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2023). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *arXiv preprint arXiv:2310.11511*.

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Osazuwa Ness, R., & Larson, J. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130*.

Guo, Z., Xia, L., Yu, Y., Ao, T., & Huang, C. (2024). LightRAG: Simple and Fast Retrieval-Augmented Generation. *arXiv preprint arXiv:2410.05779*.

Izacard, G., & Grave, E. (2021). Leveraging Passage Retrieval with Generative Models for Open-Domain Question Answering. *arXiv preprint arXiv:2007.01282*.

Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *arXiv preprint arXiv:2004.04906*.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, 33, 9459–9474.

Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., & Manning, C. D. (2024). RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. In *International Conference on Learning Representations (ICLR)*.

Yan, S., Gu, J., Chen, Y., & Zhang, H. (2024). Corrective Retrieval Augmented Generation. *arXiv preprint arXiv:2401.15884*.

Zhuang, L., Chen, S., Xiao, Y., Zhou, H., Zhang, Y., Chen, H., Zhang, Q., & Huang, X. (2025). LinearRAG: Linear Graph Retrieval-Augmented Generation on Large-Scale Corpora. *arXiv preprint arXiv:2510.10114*.